

Recursion



Good

Morning

- Today's content
- Recursion Introduction
 - Function call Tracing
 - Factorial of No.
 - Increasing
 - Decreasing
 - Fibonacci no.

Recursion $\rightarrow$ Function calling itself

$\rightarrow$ Repeatable task

$\rightarrow$ Solve a problem by using a subproblem

a smaller instance
of same problem

* Sum of first 5 natural numbers

$$\text{sum}(5) = 1 + 2 + 3 + 4 + 5$$

$$\text{sum}(4) = 1 + 2 + 3 + 4$$

$$\text{sum}(5) = \underbrace{\text{sum}(4)}_{\text{subproblem}} + 5$$

$$\text{sum}(N) = \underbrace{1 + 2 + 3 + \dots + (N-1)}_{\text{sum}(N-1)} + N$$

$$\text{sum}(N) = \text{sum}(N-1) + N$$

* How to write recursive Code

01. Expectations $\rightarrow$ Decide what our function is suppose to do.
02. Main logic $\rightarrow$ Breaking main problem into subproblem & then using it to get final answer.
03. Base Case $\rightarrow$ Last valid input for which recursion needs to stop.

Q Write a function that calculate & return sum of N natural numbers. ($N \geq 1$)

Expectation $\rightarrow$ calculate & return sum of N

```
int sum( int N ) {  
    if ( N == 1 ) return 1;  
    int rec = sum( N-1 ) + N;  
    return rec;  
}
```

TC: $O(N)$
SC: $O(N)$

```
void main ( )  
{  
    N = 4  
    int ans = sum(N);  
    print (ans);  
}
```

10

```
int sum(int N) { N = 4
```

```
    if (N == 1) return 1;
```

```
    int rec = sum(N-1) + N;
```

6 + 4 = 10

```
    return rec;
```

```
int sum(int N) { N = 3
```

```
    if (N == 1) return 1;
```

```
    int rec = sum(N-1) + N;
```

3 + 3 = 6

```
    return rec;
```

```
int sum(int N) { N = 2
```

```
    if (N == 1) return 1;
```

```
    int rec = sum(N-1) + N;
```

1 + 2 = 3

```
    return rec;
```

```
int sum(int N) { N = 1
```

```
    if (N == 1) return 1;
```

```
    int rec = sum(N-1) + N;
```

```
    return rec;
```

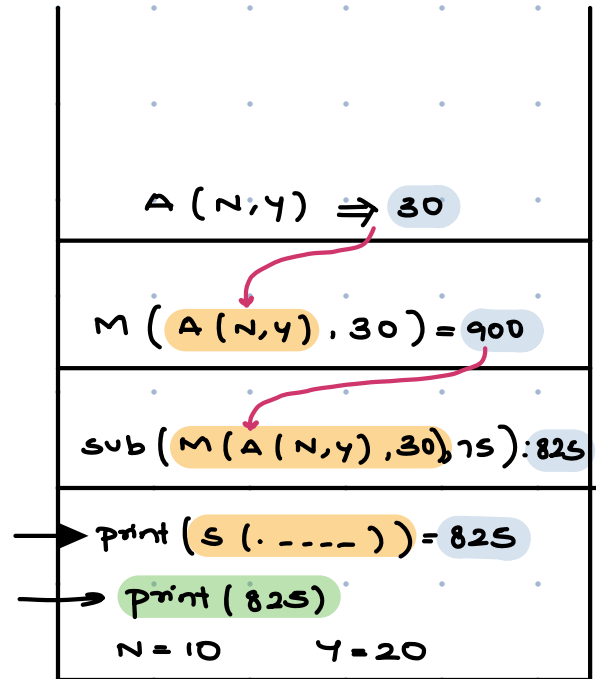
* Function Call tracing

Code-block

```
int add ( int x, int y ){  
    return x + y ;  
}  
int mul ( int x, int y ){  
    return x * y ;  
}  
int sub ( int x, int y ){  
    return x - y ;  
}  
void print ( int x ){  
    print (x) ;  
}
```

Program

```
1 main() {  
2     n = 10, y = 20  
3     print( sub( mul( add(n, y), 30 ), 75 ) )  
}
```



Conclusion

- Function call will go & get stacked at the top.
- The child call will go & return its answer to parent.
- Parent will wait for the children to return their answer before moving forward.

Q Given a non negative no. N , find factorial of N ($N \geq 0$)

$$\text{fact}(5) = 5 * \underbrace{4 * 3 * 2 * 1}_{= 120} = 120$$

$$\text{fact}(5) = 5 * \text{fact}(4)$$

$$\text{fact}(N) = N * \underbrace{(N-1) * (N-2) * \dots * 2 * 1}_{= \text{fact}(N-1)}$$

$$\text{fact}(N) = N * \text{fact}(N-1)$$

```
int fact (int N)
```

```
    if (N == 0) return 1;
```

```
    int rec = fact(N-1);
```

```
    return N * rec;
```

```
}
```

$$0! = 1$$

TC: $O(N)$

SC: $O(N)$

```
void main() {
```

```
    ans = fact(4)  
          24
```

```
}
```

```
int fact (int N)  $N=4$ 
```

```
    if (N == 0) return 1;
```

```
    int rec = fact(N-1);  
               6
```

```
    return N * rec;  
           4 * 6
```

```
}
```

```
int fact ( int N ) N=3
```

```
if ( N==0 ) return 1 ;
```

```
int rec = fact(N-1);
```

```
return N * rec;
```

```
}
```

```
int fact ( int N ) N=2
```

```
if ( N==0 ) return 1 ;
```

```
int rec = fact(N-1);
```

```
return N * rec;
```

```
}
```

```
int fact ( int N ) N=1
```

```
if ( N==0 ) return 1 ;
```

```
int rec = fact(N-1);
```

```
return N * rec;
```

```
}
```

```
int fact ( int N ) N=0
```

```
if ( N==0 ) return 1 ;
```

```
int rec = fact(N-1);
```

```
return N * rec;
```

```
}
```

Q3 Given N , print all no. from 1 to N in increasing order.

Inc(5)

1

2

3

4

5

Inc(4)

1

2

3

4

Main logic

Inc(4)

print(5);

$\text{Inc}(5) \Rightarrow \text{Inc}(4)$
print(5);

$\text{Inc}(N) \Rightarrow \text{Inc}(N-1)$
print(N)

```
void Incre (int N) {
```

```
    if (N == 0) return;
```

```
    Incre (N-1);
```

```
    System.out.print(N);
```

```
    if (N == 1)
        SOP(N)
        return;
```

TC: $O(N)$

SC: $O(N)$

```
void Incre (int N) { N=3
```

```
    if (N==0) return;
```

```
    Incre (N-1);
```

```
    system.out.print(N);
```

```
    }
```

```
void Incre (int N) { N=2
```

```
    if (N==0) return;
```

```
    Incre (N-1);
```

```
    system.out.print(N);
```

```
    }
```

Output

1

2

3

```
void Incre (int N) { N=1
```

```
    if (N==0) return;
```

```
    Incre (N-1);
```

```
    system.out.print(N);
```

```
    }
```

```
void Incre (int N) { N=0
```

```
    if (N==0) return;
```

```
    Incre (N-1);
```

```
    system.out.print(N);
```

```
    }
```

Ques $\Rightarrow$ What if we have to print from N to 1

```
void Decr (int N) { ... }
```

* Print Array using recursion

$A[] = \{ \underset{0}{1} \ \underset{1}{2} \ \underset{2}{3} \ \underset{3}{4} \ \underset{4}{5} \}$

output = 1 2 3 4 5

```
void printAr ( int [ ] A, int 0idx)
{
    if (idx == N) return;
    System.out.print ( A[idx]);
    printAr ( A, idx+1);
}
```

TC : $O(N)$

SC : $O(N)$

* Time & Space complexity for Recursion

Time complexity = Time taken by our recursive code

Space complexity = Amount of space taken by
function call in stack

Time Complexity = $O(\text{Number of function calls} * \text{Time per function call})$

Space Complexity = $O(\text{Maximum depth of recursion tree/stack space} * \text{Space per function call})$

```
int fact ( int N )
```

```
{
    if ( N == 0 ) return 1;
```

```
int rec = fact(N-1);  
return N * rec;  
}
```

$T(N)$ = Total time taken by fact(N)

$T(N-1)$ = Total time taken by fact(N-1)

$$T(N) = T(N-1) + 1$$

$$T(0) = 1$$

} Recurrence relation

$$T(N) = \underline{T(N-1)} + 1 \longrightarrow 1^{\text{st}} \text{ equation}$$

$$T(N-1) = T(N-1-1) + 1$$

$$\underline{T(N-1)} = T(N-2) + 1$$

$$T(N) = T(N-2) + 1 + 1$$

$$T(N) = \underline{T(N-2)} + 2 \longrightarrow 2^{\text{nd}} \text{ equation}$$

$$\underline{T(N-2)} = T(N-3) + 1$$

$$T(N) = T(N-3) + 1 + 2$$

$$T(N) = T(N-3) + 3 \longrightarrow 3^{\text{rd}} \text{ equation}$$

After k substitution

$$T(N) = T(N-k) + k$$

$$T(0) = 1$$

$$N-k=0$$

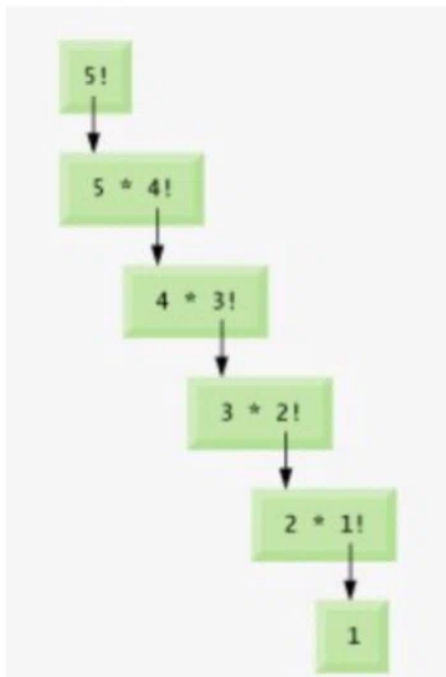
$$N=k$$

$$T(N) = 1 + k$$

$$= 1 + N$$

$$T(N) = O(N)$$

Recursive Tree for Factorial



$$SC: O(N)$$

* Fibonacci Series

N : 0 1 2 3 4 5 6 7

fib(N) : 0 1 1 2 3 5 8 13

fib(N) = sum of previous two fibonacci no.

$$\text{fib}(5) = \text{fib}(4) + \text{fib}(3)$$

$$\text{fib}(N) = \text{fib}(N-1) + \text{fib}(N-2)$$

```
int fib (int N){
```

```
    if (N <= 1) return N;
```

```
    return fib(N-1) + fib(N-2);
```

```
}
```

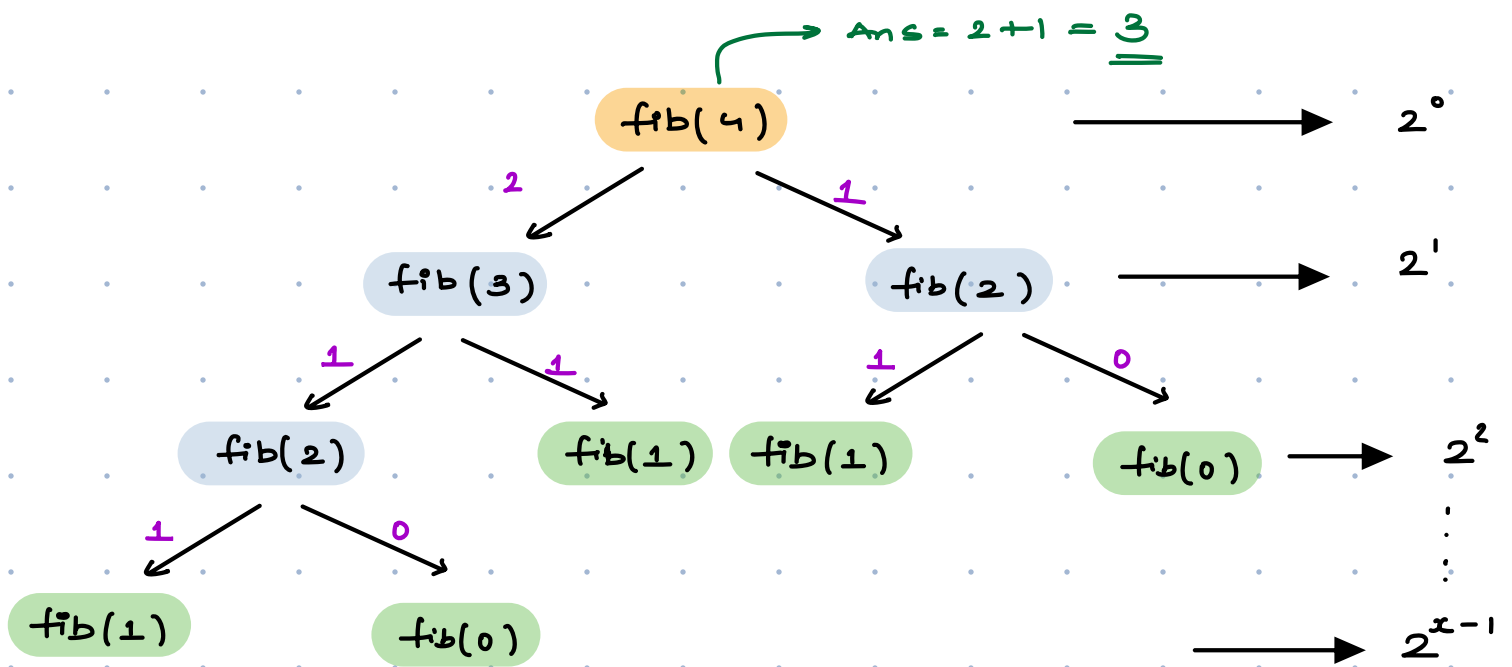
if (N==0) return 0
if (N==1) return 1

Base case $\rightarrow$ last valid input for which we know the ans
& we want recursion to stop

$$N=0 \rightarrow \text{fib}(-1) + \text{fib}(-2)$$

$$N=1 \rightarrow \text{fib}(0) + \text{fib}(-1)$$

$$N=2 \rightarrow \text{fib}(1) + \text{fib}(0)$$



On level 1 = 2^0

On level 2 = 2^1

on level 3 = 2^2

⋮

On level x = $2^{x-1} \Rightarrow 2^{N-1}$
 last level

x & N are same

$$\text{Total calls} = 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^{N-1}$$

$$\text{sum} = \frac{a(r^N - 1)}{r - 1} \Rightarrow \frac{2^0 * (2^N - 1)}{2 - 1}$$

$$\Rightarrow 2^N - 1$$

$$T_C: O(2^N * 1)$$

$$T_C: O(2^N)$$

$$S_C: O(N)$$